

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

Съдържание:

1. Erlang и Elixir

1.1. Erlang.

1.2. Elixir.

1.2.1. Еликсир функции.

2. Характеристика и особености на BEAM виртуална машина.

3. Разпределение на паметта, начин на извод.

3.1. Въведение.

3.2. BEAM виртуална машина.

3.2.1. Области на данни.

3.2.2. Обекти на данни.

3.2.3. Регистри.

3.2.4. Извиквания на функции.

3.2.5. Едновременност.

3.3. Инструкции за виртуална машина BEAM.

3.3.1. Формат на инструкции.

3.3.2. Инструкции за форматиране.

3.3.3. Процедурни инструкции.

3.3.4. Инструкции за поставяне (put).

3.3.5. Инструкции за получаване (get).

3.3.6. Test инструкции.

3.3.7. Инструкции за преместване (move).

3.3.8. Обработка на изключения.

3.3.9. Междупроцесна комуникация.

3.3.10. Примитиви.

3.3.11. Вградени функции.

3.3.12. Аритметични изрази.

3.3.13. Сравнения на срокове.

3.3.14. Инструкции за индексирание.

3.3.15. Сгъване на инструкции.

1. Erlang и Elixir

1.1.Erlang

Erlang (Ерланг) е програмен език и среда за изпълнение, създаден за изграждане на паралелни и разпределени изчислителни системи. През 1986 г. е разработена първата версия на Erlang от Ericsson Computer Science Laboratory (в разработката участват Джо Армстронг, Робърт Вирдинг и Майк Уилямс). Първоначално езикът е разработен за използване в няколко големи телекомуникационни системи на Ericsson, където от съществено значение са били високата производителност и надеждност.

Ерланг комбинира идеи от функционалното програмиране с техники за изграждане на отказоустойчиви системи, за да предостави средство, с чиято помощ по-лесно да се разработват паралелни приложения. Т.е. езикът е построен по такъв начин, че да има интегрирана поддръжка за паралелност, разпределение и толерантност към грешки.

С цел популяризиране на проекта извън Ericsson, през 1998 г. се пуска с отворен код, заедно с ОТР, колекция от мидълуер и библиотеки в Erlang. Днес се поддържа от продуктовата единица ОТР в Ericsson като са широко наричани Erlang / ОТР.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

Предимства на Erlang:

- програмите на Erlang, за да използват цялата сила на многоядрените процесори, са съставени от милиони малки процеси;
- **приложенията работят почти четири пъти по-бързо на четириядрен процесор, отколкото на едноядрен, без да се налага да се правят промени;**
- Erlang притежава **нетипичен синтаксис;**
- **за поддръжка на паралелност се следва Actor модела, при който процесите си комуникират чрез съобщения;**
- **спестява трудностите при работа с нишки и синхронизация;**
- **предоставя нестандартно решение за използване на целия капацитет на многоядрените процесори;**
- **Ерланг са алтернативно решение срещу сложността и високата цена за разработка на паралелни приложения;**
- **Erlang дълго време се използва за разработка на големи системи, които работят в продължение на години с минимално количество прекъсвания;**
- **предоставя лесна мащабируемост и надеждност, като се използва за критични откъм натовареност части на уеб приложения и уеб услуги.**

Примери за Ерланг приложения са: Facebook Chat, Ejabber, Amazon SimpleDB, Apache CouchDB, RabbitMQ.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

1.2. Elixir

Elixir е динамичен, функционален език, създаден за изграждане на мащабируеми и поддържаеми приложения. Той използва виртуалната машина Erlang, известна със своите разпределени и устойчиви на грешки системи на ниско ниво, и се използва успешно в уеб разработката и вградения софтуер. Elixir, написан от Jose Walim, разширява Erlang и осигурява приятелски синтаксис в Erlang VM, като запазва същата производителност като Erlang.

1.2.1. Еликсир функции

Характеристики на Еликсир:

- **Мащабируемост** - целият код на Elixir работи в леки процеси, които са изолирани и обменят информация чрез съобщения.
- **Толерантност към неизправности** - Elixir предоставя надзорни органи, които описват как да бъдат рестартирани части от система, когато нещо се обърка, връщайки се в известно първоначално състояние, което гарантирано работи. Това от своя страна гарантира, че **никога няма да има срив в приложението/платформата**.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

2. Характеристика и особености на BEAM виртуална машина

BEAM е виртуална машина в основата на Erlang Open Telecom Platform (OTP). BEAM е част от Erlang Run-Time System (ERTS), която компилира Erlang и Elixir изходен код в байт код, който впоследствие се изпълнява на BEAM. Файловете с байт код BEAM имат .beam разширение на файла. Съкращението BEAM идва от Абстрактна Ерланг Машина на Богдан (кръстена на Богумил "Богдан" Хаусман, който е написал оригиналната версия), но се среща и с името Абстрактна Машина на Björn Erlang (по името на Бьорн Густавсон, който е написал и поддържа текущата версия). И двамата разработчици са работили по системата, докато са били в Ericsson. Предшественикът на BEAM е JAM, съкратено от Абстрактна Машина на Джо, която е първата виртуална машина за езика Erlang и е написана от Джо Армстронг.

Erlang е ориентиран към паралелност. Но какво е паралелност и каква е разликата между едновременност (паралелност) и паралелизъм?

Паралелността е концепцията за наличието на два или повече процеса, които могат да се изпълняват независимо един от друг, т.е. първо се изпълнява единият а след това другият процес, или се преплита изпълнението им, или процесите се изпълняват паралелно. При паралелното изпълнение, процесите се изпълняват по едно и също време, като се използват няколко физически единици за изпълнение.

Паралелизъм може да се постигне на различни нива. Чрез множество изпълнителни единици в изпълнителния конвейер в едно ядро, в няколко ядра на един процесор, от няколко процесора в една машина или чрез няколко машини.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

Erlang използва процеси за постигане на паралелност. Концептуално процесите на Erlang са подобни на повечето процеси на ОС, те се изпълняват паралелно и могат да комуникират чрез сигнали. На практика има огромна разлика в това, че процесите на Erlang са много по-леки от повечето процеси на други ОС. Много други конкурентни езици за програмиране наричат своите еквиваленти на Erlang процеси *агенти*. Erlang постига паралелност чрез преплитане на изпълнението на процеси на виртуалната машина Erlang, BEAM.

Многоядреният процесор BEAM може да постигне паралелизъм, като стартира по един **планировчик на ядро** и изпълнява един процес на Erlang на планировчик. Дизайнерът на система Erlang може да постигне допълнителен паралелизъм, като разпредели системата на няколко компютъра.

Типична система Erlang (сървър или услуга, вградена в Erlang) се състои:

- от множество *приложения* на Erlang, съответстващи на директория на диска.
- всяко приложение се състои от няколко *модула* Erlang, съответстващи на файлове в директорията.
- всеки модул съдържа редица *функции* и всяка функция се състои от *изрази*.

Като функционален език Erlang няма твърдения, а само изрази.

- изразите на Erlang могат да се комбинират във функция на Erlang.
- функцията приема редица аргументи и връща стойност.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

Пример за Erlang Code - `%% Some Erlang expressions: true.`

`1+1.`

`if (X > Y) -> X; true -> Y end.`

`%% An Erlang function:`

`max(X, Y) ->`

`if (X > Y) -> X;`

`true -> Y`

`end.`

Erlang има редица *вградени функции* (или *BIF*), които се изпълняват от Virtual Machine. Възможно е да се осигури функционалност на ниско ниво, което по принцип е трудно или невъзможно да се приложи в самия Erlang, като `list_to_atom`. Може също така да бъдат предоставени собствени функции, внедрени в C, като се използва интерфейса за *реализирани функции* (*NIF*), както в Erlang / OTP R13B03.

3. Разпределение на паметта, начин на извод

Езикът за програмиране Elixir обхваща функционално програмиране и работи на базата на индустриална, високопроизводителна, разпределена Erlang Virtual Machine.

Какво означава това?

Не е нужно да се мисли много относно защитата на последователността на данни в многонишкова среда. По-лесно е да се мащабират създаваните приложения.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

3.1. Въведение

Ще бъдат разгледани инструкции на Erlang BEAM, използвани за емуляция на нишков код на BEAM. BEAM ще бъде описан такъв, какъвто е бил през 1997 г.

3.2. BEAM виртуална машина

3.2.1. Области на данни

Областите с данни са кодовата област, съдържаща зареден компилиран код на Erlang, както и stack (стек) и heap. Стекът съдържа рамки за извикване с локални променливи и адреси за връщане. Heap-а съдържа условия, създадени от изпълнението на Erlang. Стекът и heap са разпределени като една област на паметта и нарастват една към друга. Наличието на heap и стек заедно прави тестването за препълване на паметта много ефективно, тъй като се сравняват двата указателя (указател „top-of-heap“ т.е. от върха на heap и указател „top-of-stack“ т.е. от върха на стека), намиращи се в хардуерните регистри.

Всеки кадър за извикване в стека започва с указател за връщане на адреса, CP, последван от локални променливи, до които се осъществява достъп чрез целочислено изместване от показалец към върха на стека. Рамките се разпределят само след като клаузата се оцени и ако тялото на клаузата съдържа локални променливи и извиквания на функции.

Кадрите се разпределят и изхвърлят от A_or и D_or инструкции с изрично зададен размер на кадъра N. За целите на събирането на боклука, структурата на стека може да бъде получена чрез търсене на съхранени адреси за връщане.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

Рамката за повикване може да съдържа също адреси за възобновяване (Catch инструкция) и указатели за следваща инструкция (инструкция C).

Системата Erlang BEAM позволява смесване на емуляция с нишков код с компилиране в C. Когато се извика компилирана на C Erlang функция, следващият указател за инструкция I се съхранява в текущата локална рамка (кадър) и CP указателят е настроен на кода на емулятора, което би възстановило указателя I при връщане от извиканата функция.

3.2.2 Обекти на данни

Erlang term е представен от 32-битова unsigned word, съдържаща стойност и маркер.

Етикетът е 4-битов и се намира в най-малко значимите битове и разграничава вида на term.

Стойността на атома е индекс в глобална таблица с атоми, където е представен атомът.

Стойността на „small integer“ е самото цяло число.

Стойността на „big number integer“ е указател към heap обект, съдържащ целочисления знак и броя на думите, изграждащи целочислената стойност.

Стойността на списък е указател към две последователни местоположения на heap с два маркирани обекта (главата и опашката на списъка).

Стойността на кортеж е указател към heap обект, съдържащ размера на кортежа, последван от елементите му.

Стойността на „float“ е указател към heap обект, съдържащ плаваща стойност с две думи.

Стойността на идентификатора на процеса е самият идентификатор на процеса.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

Таблица 1: Текущата реализация на BEAM използва следните тагове:

#define ARITYVAL	10	/* tuple arity */
#define ATOM	7	/* atom */
#define BIG	11	/* big integer pointer */
#define BINARY	14	/* binary pointer */
#define BLANK	ARITYVAL	/* blank local var */
#define CATCH	THING	/* resumption address */
#define CP0	0	/* CP pointer */
#define CP12	12	/* CP pointer */
#define CP4	4	/* CP pointer */
#define CP8	8	/* CP pointer */
#define FLOAT	9	/* float pointer */
#define IC	SMALL	/* next instr. pointer */
#define LIST	1	/* list pointer */
#define MOVED	12	/* moved heap pointer */
#define NIL	(BIG+16)	/* empty list */
#define PID	3	/* process identifier */
#define PORT	5	/* port */
#define REFER	6	/* reference */
#define SMALL	15	/* small integer */
#define THING	13	/* float value */
#define TUPLE	2	/* tuple pointer */

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

Тъй като някои обекти от данни могат да се намират само в heap-а или стека, един и същ маркер може да се използва за два различни обекта (*CATCH* и *THING*, *BLANK* и *ARITYVAL*). Има и други случаи, когато един и същ маркер може да се използва повторно. Тъй като обектите, представляващи големи цели числа, са маркирани указатели на heap-а, флагът *BIG* може да се използва и за представяне на празен списък, *NIL* (тъй като частта със стойност *NIL* става невалиден указател на heap-а). *CP* таговете са запазени за връщане на адреса, *CP*. Указателят не трябва да бъде маркер, ако на текущата платформа на указателят винаги последните два бита са зададени да са 0.

3.2.3. Регистри

Таблица 2. Регистри използвани във BEAM

Наименование	Описание
HTOP	указател на върха на heap-а
E	указател на върха на стека
CP	указател за връщане на адреса (къде да отидете, когато дадена функция е готова за изпълнение)
I	указател за следваща инструкция
x(N)	регистри на аргументите (за предаване на параметрите на функцията), използват се също за съхраняване на времеви променливи
y(N)	локални променливи (y(N) не са реални регистри, те се намират в локална рамка и са достъпни чрез цялостно изместване от указател отгоре на стека)
fcalls	брой направени намаления (за да се прави проверка за спиране, докато се извиква дадена функция)

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

BEAM е регистърна машина, където всички инструкции работят върху поименни регистри. Всеки регистър може да съдържа всеки `term Erlang` като цяло число или кортеж и помага да се мисли за тях като за прости променливи. Двата най-важни регистра са:

$x(N)$: те се използват за временни данни и предаване на данни между функции. Те не изискват рамка на стека и могат да се използват свободно във всяка функция, но имат някои ограничения;

$y(N)$: те са локални за всяка рамка на стека и нямат специални ограничения, освен нуждата от рамка на стека.

3.2.4 Извиквания на функции

За извикване на Erlang функция, се предават параметри, като се зареждат регистрите на аргументи, $x(N)$, и се актуализира регистър за връщане на адреса, CP . При връщане на функцията връщаната стойност се съхранява в $x(0)$.

3.2.5 Едновременност

Процесите на Erlang се създават динамично и се убиват по време на изпълнението. Всеки процес има свой собствен heap и област на стека. За целите на едновременното използване на Erlang, BEAM осигурява механизми за спиране и планиране. Процесът на спиране съхранява текущото си състояние в своя запис за спиране и се добавя към опашката за планиране. За да се гарантира справедливо планиране, процесът се спира след определен брой итерации и след това първият процес от опашката се възобновява.

За получаване на съобщения всеки процес има своя собствена опашка за локални съобщения. Изпращането на съобщение води до копиране на съобщението в heap-а на приемника и съхраняване на препратката към съобщението в опашката за съобщения на приемника. Докато чакат съобщенията да бъдат получени, процесът се разменя и се добавя към опашката за планиране само, когато се получи ново съобщение (т.е. добавянето се извършва чрез процес, изпращащ съобщението), или настъпва изчакване.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

3.3. Инструкции за виртуална машина BEAM

Описаният набор от инструкции на Erlang BEAM, се използва за емуляция на кодове с нишки. Ще бъде разгледан форматът на всяка от посочените инструкции:

- ☐ за форматиране;
- ☐ процедурни;
- ☐ за поставяне;
- ☐ за вземане;
- ☐ за изпитване;
- ☐ за преместване;

също така:

- ☐ обработка на изключения;
- ☐ междупроцесна комуникация;
- ☐ примитиви;
- ☐ вградени функции.

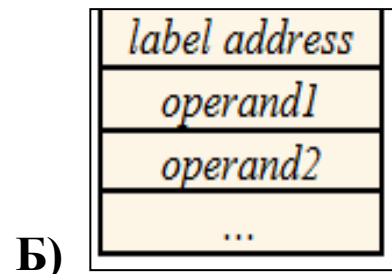
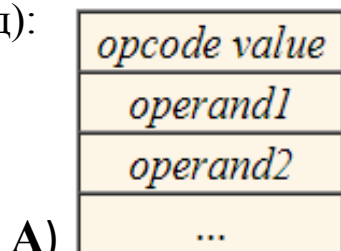
3.3.1 Формат на инструкциите

Синтаксис с байтов код (произведен от компилатора BEAM и записан във файла **module_name.beam**

Синтаксис с нишков-код (произведен от BEAM зареждача при зареждане на компилираният байт код):

Фиг.1. Синтаксис на инструкции

А) байт код;
Б) нишков код



Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

<i>opcode value</i>
<i>operand1</i>
<i>operand2</i>
...

Синтаксис с байтов код

Ако стойностите на **opcode** са по-малки от 255 се кодират, като един байт, ако стойностите на **opcode** са по-големи от 255 се кодират, като два байта (255 и останалите).

Модули, функции, имена на атоми и числа с плаваща запетая се кодират като списък с ASCII стойности, предшествани от дължината на списъка. Т.е. операндите в синтаксис диаграма на байтов код според типа им имат различен размер:

8-bit long

magic number
function arity
type of arithmetic
type of operand
type of bif
sign of bignumber value

16-bit long

atom length
float length
code length
register number
frame size
heap requirement

24-bit long

relative address

32-bit long

integer value
code size
tuple arity
tuple index
string length
bignum arity
bignum value
други операнди са с
дължина 16 бита.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

label address
operand1
operand2
...

Синтаксис с нишков код

Адресът на етикета (четири байта) е адрес на съответния код на емулятора. След зареждането атомите и малките цели числа са готови (индекс в таблицата с атоми + TAG, цяло число + TAG). След зареждането всички операнди са с дължина четири байта (една 32-битова дума). Когато операнд е NIL или времева променлива $x(0)$, се използва само една дума. Когато операнд е времева променлива $x(N)$ или локална променлива $y(N)$, първата дума показва типа на променливата (X_arg или Y_arg), а втората - нейния индекс N .

За постигане на ефективност, в много инструкции, типовете операнди са неявно кодирани като инструкции opcode. Всеки ред в синтаксисната диаграма с нишков код представлява 32-битова дума.

Когато синтаксисът с кодове с нишки съответства на синтаксисът на байтов код, тогава само кодът на инструкцията за операция се променя на адрес на етикет и се изграждат операнди, като се добавя &FuncBegin или изместването на инструкцията се променя на адрес на инструкция за целите на разклонението.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

IntCode_op
code size
module name
magic number

3.3.2. Инструкции за форматиране

Предоставя информация на loader (товарач) с нишкови кодове.

Представен със синтаксис с байтов код

☐ *IntCode*

Синтаксис с нишков код: няма

(инструкцията се премахва от loader (товарач))

code size (размерът на кода) е общ размер (в 32-битови думи) на компилирания код на модула след зареждане

module name е име на атом, съответстващо на името на модула

magic number (магическо число) се използва за предотвратяване на зареждането на код на стар формат (проверява се с магическо число на loader при зареждане)

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

FuncBegin_op
function_name
function_arity

lb_FuncBegin
module_name
function_name
function_arity

3.3.3. Процедурни инструкции

❑ *FuncBegin*

Генерира грешка *BADMATCH* (има лошо съвпадение в *clause/case/if/receive* Erlang).

Представен със синтаксис с байтов и нишков код

module_name, function_name са готови създадени атоми

Генерира текущата контекстна информация (текущия модул, функция и *arity*), задава *BADMATCH* грешка, клон към процедура за обработка на грешки.

Инструкцията *FuncBegin* стартира всяка компилирана Erlang функция. За да се спести място, предоставената контекстна информация се използва повторно от други инструкции. Когато се извика функция Erlang, се изпълнява първата инструкция след *FuncBegin*.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

C/CO/ResC_op
function name
function arity

lb_C/CO/ResC
&function
function_arity
&FuncBegin

❑ C/CO/ResC

Представен със синтаксис с байтов и нишков код

C: Извиква локална Erlang функция. Локална функция е тази, която се намира в същия модул като функцията за извикване.

CO: Извиква локална Erlang функция. Указателят за продължаване на програмата (CP регистър) вече е зададен.

ResC: Извиква резидентна Erlang функция. Резидентна функция Erlang е локална функция, която не може да бъде извикана и да се върне към други модули. Инструкцията се използва за оптимизиране на управлението на контрола, когато Erlang функция с кодова нишка не може да се смесва с други функции на Erlang, компилирани в C код.

&function е адрес на първата инструкция, съответстваща на извикваната функция (по точно първата инструкция след съответния FuncBegin).

function_arity е номер на функцията аргументи.

&FuncBegin е адрес на текущата функция FuncBegin инструкция.

C: Съхранява указател към следващата инструкция в текущата рамка на стека. Задава указател на програмата за продължаване в кода на емулятора, като възстановява следващия указател на инструкция, увеличава брояча на редукции и проверява за прекъсване (ако има прекъсване, генерира текущия модул, функция, както и информация за arity с помощта на &FuncBegin), разклонява се към извиканата функция.

CO: Както по-горе, но указателят на програмата за продължаване вече е зададен в регистъра на CP (инструкцията се използва, когато се извика последната функция в тялото и няма рамка от стека, която да бъде изхвърлена).

ResC: Задава указател на програмата за продължаване при следващата инструкция, увеличава брояча на редукции и проверява за прекъсване (ако прекъсне, генерира текущия модул, функция, както и информация за arity с помощта на &FuncBegin), разклонява се към извиканата функция.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

CL_op
function name
function arity
frame size

lb_CL
&function
function_arity
frame_size
&FuncBegin

□ *CL*

Изхвърля текущата рамка на стека. Извиква локална Erlang функция.

Представен със синтаксис с байтов и нишков код

frame_size е размер на текущата рамка на стека, която трябва да се изхвърли.
(Останалите операнди са сходни с тези от инструкцията C/CO.)

Копира указател на програмата за продължаване от текущата рамка на стека в CP, изчиства рамката, увеличава брояча на редукции (reductions) и проверява за прекъсване, разклонява се към извиканата функция.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

TrC/TrCO_op
function name
function arity

lb_TrC/TrCO
&function
function_arity
&FuncBegin
function_name

□ **TrC/TrCO**

Представен със синтаксис с байтов и нишков код

TrC: Проследена версия на инструкцията C.

TrCO: Проследена версия на CO инструкцията.

function_name е атом, съответстващ на извиканото име на функция; използвани за генериране на информация за проследяване.

(Другите операнди могат да се видят на C/CO.)

Същата семантика като C/CO, но известна информация за проследяване (наричаният функционален модул, име, arity) се изпраща до проследяващ Erlang процес.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

TrCL_op
function name
function arity
frame size

lb_TrCL
&function
function_arity
frame_size
&FuncBegin
function_name

❑ *TrCL*

Представен със синтаксис с байтов и нишков код

TrCL е проследена версия на *CL* инструкцията.

function_name е атом, съответстващ на извиканото име на функция; използвани за генериране на информация за проследяване.

(Другите операнди са сходни с TrCL)

Същата семантика като CL, но известна информация за проследяване (наричаният функционален модул, име, arity) се изпраща до проследяващ Erlang процес.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

CEx/TrCEx_op
module_name
function_name

lb_CEx/TrCEx
module_name
function_name
function_arity

□ *CEx/TrCEx*

Извиква външна Erlang функция. TrCEx е проследена версия на инструкцията CEx.

Представен със синтаксис с байтов и нишков код

module_name, **function_name**, **function_arity** дефинира външната функция за извикване.

&FuncBegin е адрес на текущата функция FuncBegin инструкция.

CEx: Задава указател на програмата за продължаване към следващата инструкция, увеличава брояча на редукции и проверява за прекъсване (ако прекъсне, генерира текущия модул, функция както и информация за arity с помощта на &FuncBegin), проверява дали извиканата функция е заредена, ако **условието е изпълнено**, се разклонява към наречена функция (адресът е взет от таблицата с функции за експортиране), **ако пък не е** се извика манипулатор на грешки.

TrCEx: Същата семантика като CEx, но известна информация за проследяване (наричаната функция: модул, име, arity) се изпраща до проследяващ Erlang процес.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

CExL/TrCExL_op
module name
function name
function arity
frame size

lb_CExL/TrCExL
module_name
function_name
function_arity
&FuncBegin
frame_size

□ CExL/TrCExL

Изхвърля текущата рамка на стека. Извиква външна Erlang функция. TrCExL е проследена версия на инструкцията CExL

Представен със синтаксис с байтов и нишков код

frame_size е размер на текущата рамка на стека, която трябва да се изхвърли.

(Други операнди са сходни с CEx/TrCEx).

CExL: Копира указателя на програмата за продължаване от текущия стеков кадър в CP, изхвърля рамката, увеличава брояча на редукции и проверява за прекъсване, проверява дали е заредена извиканата външна функция, разклонява се към функцията.

TrCExL: Същата семантика като CExL, но известна информация за проследяване (наричаната функция: модул, име, arity) се изпраща до проследяващ процес Erlang.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_R/ResR

□ *R/ResR*

R: Връщане от извиквана Erlang функция.

ResR: Връщане от местно повикване на Erlang функция.

Синтаксис с нишков код

R: Възстановя следващия указател на инструкция от текущата рамка на стека. Връща контрола на повикващия функцията.

ResR: Прехвърляне към инструкция, чийто адрес се съхранява в CP регистъра.

lb_TrR
& FuncBegin

□ *TrR*

Проследено връщане от извикана Erlang функция.

Синтаксис с нишков код

Същата семантика като R, като известна информация за проследяване (функцията, към която се връщаме: модул, име, arity) се изпраща до проследяващ Erlang процес.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_ERR_ACT
& FuncBegin

❑ *ERR_ACT*

Генерира грешката FUNCTION_CLAUSE (няма Erlang клаузи, съответстващи на даденото извикване на функцията).

Синтаксис с нишков код

Генерира текущата контекстна информация с помощта на указателя &FuncBegin, задава грешката FUNCTION_CLAUSE, разклонява се към процедура за обработка на грешки.

lb_A
frame_size
n

❑ *A*

Разпределя нова рамка на стека.

Синтаксис с нишков код

n е брой живи временни променливи $x(0), \dots, x(n-1)$ (за събиране на боклук). Проверява за необходимо място в стека, извиква събирача на боклук, ако е необходимо. Разпределя нова рамка на стека (на frame_size). Актуализира горната част на върха на стека - указателя E. Копира указателя за продължаване на програмата CP в новия кадър.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_I
n

□ *I*

Инициализира локална променлива (намираща се в текущата рамка на стека).
Синтаксис с нишков код

n - индекс на локална променлива $y(n)$, която трябва да бъде изрично инициализирана.

Инициализира локална променлива $y(n)$.

lb_D
frame_size

□ *D*

Изхвърля текущата рамка на стека.
Синтаксис с нишков код

Копира указателя на програмата за продължаване от текущата рамка на стека в *CP*, изхвърля рамката. Актуализира върха на стека - указател *E* .

□ *TH*

Проверява за необходимо пространство за heap.
Синтаксис с нишков код

m е размер на необходимото пространство за heap (с думи).

n е брой живи временни променливи $x(0), \dots, x(n-1)$ (за събиране на боклук).

Проверява за необходимото пространство за heap, извиква събирача на боклук, ако е необходимо.

lb_TH
m
n

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_PL
type
head
type
tail
type
list

3.3.4. Инструкции за поставяне (put)

Инструкциите за поставяне се използват за изграждане на нови обекти от данни Erlang. **Всички обекти са изградени върху купчина процес Erlang. Във всички случаи изискванията за куп пространство вече са проверени.**

□ *PL*

Наличието на head и tail създава нов списък.

Синтаксис с нишков код

type показва вида на следния операнд:

X_arg - времева променлива,

Y_arg - локална променлива,

C_arg - константа (atom, NIL или small integer).

В зависимост от типа си, операнда може да бъде индекс (за X_arg и Y_arg) или готов създаден обект от данни Erlang (за C_arg).

Наличието на head и tail създава нов списък. Задава list към новосъздадения обект от данни. Актуализира heap-top указателя, НТОР.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_PL_<type>
list
head
tail

□ *PL_<type>*

Наличието на head и tail създава нов списък. *PL_<type>* е специализирана версия на *PL* инструкцията.

Синтаксис с нишков код

PL_<type> има същата семантика като PL инструкцията. Единствената разлика е, че типовете на всички операнди се кодират имплицитно, като код за инструкции. Например PL_хух означава, че първият и третият операнди са временни променливи, докато вторият операнд е локална променлива. Решението, кои видове се кодират като специализирани инструкции се основава на тяхната честота в съществуващите големи промишлени приложения.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_PS
string_length
string_elements0
...
string_elementsN
type
list

❑ *PS*

Конвертиране на string до съответния list. Изграждане на списък.
Синтаксис с нишков код

string_length е дължината на включения низ (брой елементи на низ).

string_elements <i> е 32-битова дума, състояща се от четири низови елемента (всеки с дължина 8-бита). В зависимост от дължината на низа последните елементи от низа могат да бъдат по-къси.

type показва нещо като следващ операнд.

Като се разполага с низ с определена дължина, можете да се изгради нов списък, състоящ се от низовите елементи. Може да се зададе списък към новосъздадения обект от данни и да се актуализира heap-top указател, НТОР.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_PInt_<type>
pointer
arity
sign

□ *PInt_<type>*

Инструкцията *PInt_<type>*, последвана от редица инструкции на *PIntVal*, се използва за изграждане на голямо число.

Синтаксис с нишков код

pointer е указател към създаваното голямо цяло число.

arity дава редица **от думи с голяма стойност**.

sign е знакът за голямо число.

Може да се създаде първоначално 1 структура за голямо число. Инструкцията след това е последвана от *arity*, брой *PIntVal* инструкции. Като се зададе указател към новосъздадения обект от данни, може да се актуализира указателя *heap-top*. Типът от указателя се кодира имплицитно като код на инструкция *opcode*.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_PIntVal
value

❑ *PIntVal*

Изграждане на голяма стойност.

Синтаксис с нишков код

value е дума, която е част от голямо цяло число.

Поставете стойностна за дума в heap процеса. Инструкцията трябва да бъде предшествана от инструкцията PInt_<type> или друга инструкция PIntVal.

Актуализирайте heap-top указателя.

lb_PFlt_<type>
pointer
value0

❑ *PFlt_<type>*

Изградете двоен номер с плаваща запетая.

Синтаксис с нишков код

value0 и **value1** изграждат двойно плаващата стойност.

При поставяне на value0 и value1 в heap на процеса, се задава указател към новосъздадения обект от данни и се актуализира heap-top указателя. Типът на указателя се кодира имплицитно като opcode инструкция.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_PT0_<type>
tuple
arity

□ ***PT0_<type>***

Инструкцията ***PT0_<type>***, последвана от редица инструкции на ***PTEl_<type>***, изгражда кортеж.

Синтаксис с нишков код

tuple е указател към създавания обект с данни кортеж.

arity дава номера на следващи кортежни елементи.

Създава начална структура за обект с данни кортеж. Инструкцията е последвана от **arity** брой **PTEl_<type>** инструкции. Задава указател на кортежа към новосъздадения обект. Актуализира **heap-top** указателя. Типът кортеж е неявно (имплицитно) кодиран като **opcode** инструкция.

lb_PTEl_<type>
element

□ ***PTEl_<type>***

Изгражда кортежен елемент.

Синтаксис с нишков код

element е кортежен елемент, който се поставя в **heap**-а.

Поставя кортежен в **heap** процеса. Инструкцията трябва да бъде предшествана от инструкцията **PT0_<type>** или друга инструкция **PTEl_<type>**. Актуализира **heap-top** указателя. Типът на елемента е неявно кодиран като **opcode** инструкция.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_GL2_<type>
list
head
tail

3.5 Инструкции за получаване (get)

Инструкциите за получаване се използват за съпоставяне с аргументите на функцията Erlang, връщаните стойности на функцията и променливите Erlang. Типът на съответстващия обект вече е проверен.

□ *GL2_<type>*

Като има даден списък, вземете (head) главата и (tail) опашката му.

Синтаксис с нишков код

list е указател към съществуващ списък.

head и **tail** са указатели съответно на главата и опашката.

Ако имаме даден списък, се задава указателя на главата към главата и указателя на опашката към опашката. Като типовете на всички операнди се кодират имплицитно като opcode инструкции.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_GTp_<type>
&FuncBegin
tuple
arity

□ *G*Tp_ <type>

Ако имаме определен кортеж, проверяваме неговата надеждност. Инструкцията е последвана от редица инструкции на *GTEI*.
Синтаксис с нишков код

&FuncBegin е адрес на текущата функция FuncBegin инструкция.

tuple е указател към съществуващ кортеж.

arity е необходимото arity.

Инструкцията се среща само в clause/case/if/receive тяло. Като имаме даден кортеж, проверяваме неговата надеждност. Ако не е равна на необходимото (arity), се генерира текущата контекстна информация (използвайки &FuncBegin), възниква грешка BADMATCH и се извиква процедурата за обработка на грешки. Типът кортеж се кодира имплицитно като opcode инструкции.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_GTp_head_<type>
address
tuple
arity

□ *GTp_head_<type>*

Ако имаме определен кортеж, се проверява надеждността му. *GTp_head_<type>* е специализирана версия на инструкцията *GTp_<type>*. Синтаксис с нишков код

address е адрес на клон в случай на повреда.

tuple е указател към съществуващ кортеж.

arity е необходимата arity.

Инструкцията се среща само в clause/case/if/receive глава. Като имаме даден кортеж, се проверява надеждността му. Ако не е равна на необходимото (arity), се разклонява към следващата клауза/опция, използвайки **address**. Типът кортеж се кодира имплицитно като като opcode инструкции.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_GTEl_<type>
tuple
element
index

□ *GTEl_<type>*

Като имаме даден кортеж, вземаме неговия елемент.

Синтаксис с нишков код

tuple е указател към съществуващ кортеж.

element е кортежен елемент с необходимия индекс.

Взема се кортежен елемент с необходимия индекс. Инструкцията трябва да бъде предшествана от инструкцията `GTr_<type>` или `GTr_head_ <type>`. Типовете кортежи и елементи се кодират имплицитно като `opcode` инструкции.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_ <Test0> _ <type>
&FuncBegin
object

3.3.6. Test инструкции

Инструкциите за тестване се използват за тестване на типове от Erlang обекти от данни.

□ *<Test0>_<type>*

Проверява дали даден обект от данни Erlang има необходимия тип данни. Инструкцията обикновено предхожда други инструкции за получаване.

<Test0> е успешен, ако обект на данни е:

TNil - празен списък

TT - кортеж

TNonEmpL - непразен списък

Синтаксис с нишков код

&FuncBegin е адрес на текущата функция FuncBegin инструкция.

object е съществуващ обект от данни на Erlang

Инструкцията се среща само в clause/case/if/receive тяло. Ако имаме даден обект от данни, се проверява неговия тип. В случай на неуспех, се генерира текущата контекстна информация (използвайки &FuncBegin), изхвърля се грешката BADMATCH и се разклонява към процедурата за обработка на грешки. Типът обект (X_arg, Y_arg, C_arg) се кодира имплицитно като opcode инструкции.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_ <Test1> _head_ <type>
address
object

❑ <Test1>_head_<type>

Проверява дали даден обект от данни Erlang има необходимия тип данни. <Test1>_head_<type> е специализирана версия на инструкцията <Test0>_<type>.

<Test1> е успешен, ако обект на данни е:

TNil - празен списък

TT - кортеж

TNonEmpL - непразен списък

Синтаксис с нишков код

address е адрес на разклонение в случай на повреда.

object е съществуващ обект от данни на Erlang.

Инструкцията се среща само в clause/case/if/receive глава. Ако имаме даден обект от данни, да се провери неговия тип. В случай на неуспех, преминете към следващата клауза/опция, използвайки **address**. Типът обект (X_arg, Y_arg, C_arg) се кодира имплицитно като opcode инструкции.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_ <Test2> _ <type>
address
object

□ <Test2> _ <type>

Проверява дали даден обект от данни Erlang има необходимия тип данни. Инструкцията се среща само при защита.

<Test2> е успешен, ако обект на данни е:

- TInt - цяло число
- TFlt - float
- TCnst - константа (не е списък или кортеж)
- TL - list
- TPrId - идентификатор на процес
- TRef - reference
- TPrt - port
- TBn - binary

Синтаксис с нишков код

address е адрес на клон в случай на повреда.

object е съществуващ обект от данни на Erlang.

Инструкцията се среща само в clause/case/if/receive охрана. Ако имаме даден обект от данни, се проверява неговия тип. В случай на неуспех, се преминава към следващата клауза/опция, използвайки **address**. Типът обект (X_arg, Y_arg, C_arg) се кодира имплицитно като opcode инструкции.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_M_<type>
object
register

3.3.7 Инструкции за преместване (move)

□ *M_<type>*

Премества вътрешен обект от данни към дадена временна или локална променлива.

Синтаксис с нишков код

object е вътрешен обект с данни.

register е времева или локална променлива.

Премества вътрешен обект от данни (може да бъде от тип *X_arg*, *Y_arg*, *C_arg*) към временна или локална променлива (*X_arg* или *Y_arg*). Типовете операнди се кодира имплицитно като *opcode* инструкции.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_Throw
&FuncBegin
type
object

3.3.8 Обработка на изключения (Exception Handling)

❑ *Throw*

Изхвърляне на изключение или грешка.

Синтаксис с нишков код

object е вътрешен обект с данни

Изхвърляне на вътрешен обект от данни (може да бъде от тип X_arg, Y_arg, C_arg). Когато Throw се изпълнява, стекът се претърсва за запазен адрес за възобновяване (зададен от Catch), указателя на върха на стека се нулира, за да сочи към намерената рамка за повикване и изпълнението продължава оттам. Ако в стека няма адрес за възобновяване (бройча на процес catch е нула), текущата контекстна информация се генерира (с помощта на показалеца &FuncBegin), грешката THROWN се задава, и контролата се прехвърля в процедура за обработка на грешки.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_Catch
index
&CatchEnd

lb_CatchEnd
index

❑ *Catch*

Хвани изключение или грешка.

Синтаксис с нишков код

index е място в локалната рамка, където адреса за възобновяване &CatchEnd се запазва.

&CatchEnd е catch адреса за възобновяване.

Записване на адреса за възобновяване и CatchEnd в локалната рамка на позиция с отместване на индекса. Увеличаване на брояча catch на процеса. Инструкцията е последвана от инструкция CatchEnd. Тук се има предвид, че инструкцията CatchEnd се поставя след съответния Erlang Expression, а catch служи за предпазване от грешки.

❑ *CatchEnd*

Изчистване на адреса за възобновяване, зададен от *Catch* .

Синтаксис с нишков код

index е място в локалната рамка, където е запазен адреса за възобновяване и CatchEnd.

Изчистване на адреса за възобновяване, съхранен в локалната рамка на позиция с отместване на индекса. Намаляване на брояча catch на процеса. Инструкцията се предхожда от съпадаща инструкция Catch (за повече подробности може да се види Catch).

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_Send
&FuncBegin
type
address
type
message

3.3.9. Междупроцесна комуникация (Inter-process Communication)

❑ *Send*

Изпращане на дадено съобщение на определен адрес.

Синтаксис с нишков код

type показва вида на следния операнд (X_arg, Y_arg, C_arg).

address е адресът на изпратеното съобщение (може да бъде процес или порт на Erlang).

message е вътрешен обект от данни, съдържащ съобщението.

При наличие на дадено съобщение и то се изпраща на даден адрес. Ако адресът не е валиден, се стартира обаждане до манипулатора на грешки в Erlang. Ако докато се изпраща съобщението до отдалечен възел, съответният комуникационен канал е зает, става прекъсване за определен период от време, като се подновяват опита за изпращане на по-късен етап. При спиране се генерира текущата контекстна информация с помощта на &FuncBegin.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

❑ *Receive*

Примитивът Erlang за получаване *receive* е компилиран в поредица от инструкции, изпълняващи необходимата семантика за получаване:

```
rec: Rec (set a message pointer)
  loop: loopRec (fetch the message)
pattern: match the message against required pattern
  if matched then
    RmMsg (remove the message from the mailbox,
      mark all other messages as untried,
      if the timeout has been set then
        clear the timeout)
    evaluate given action
    TPattEnd (goto next instruction after receive)
  else
    if there are more patterns to be matched then
      goto pattern (try the next pattern)
    else
      loopRecEnd (mark the current message as tried,
        if there are more messages then
          update message pointer,
          goto loop (try the next message))
      WaitTOut (if there is given timeout and
        the timeout is not set
          set the required timeout and suspend)
      Wait (suspend)
```

Ако няма съвпадащи съобщения, процесът спира, докато не бъде получено ново съобщение или не настъпи таймаут, тогава процесът се разсрочва за изпълнение.

```
if a new message is received then
  goto rec (set a message pointer)
else if a timeout occurs then
  goto timeout (evaluate given timeout action)
else
  remain suspended
timeout: TimeOut (mark all messages as untried)
  evaluate given timeout action
  TPattEnd (goto next instruction after receive)
```

В зависимост от зададения контекст, указателят за съобщение може да означава задаване на указател към първото съобщение в пощенската кутия (когато примитивът за получаване е въведен за първи път), или задаване на указателя към първото ново получено, неразгледано съобщение (когато новото съобщение е получено и спреният процес е разсрочен за изпълнение).

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_Rec

❑ *Rec*

Задава указател за съобщение.

Синтаксис с нишков код

Инструкцията *Rec* се изпълнява, когато процес Erlang влезе за първи път в примитива за получаване (указателят на съобщението е зададен на първото съобщение в пощенската кутия) или спреният процес е пренасрочен за изпълнение (указателят е настроен на първото ново получено, неразгледано съобщение).

lb_loopRec

&wait

index

❑ *loopRec*

Изтегляне на ново съобщение.

Синтаксис с нишков код

&wait е адрес на клон в случай, че няма повече съобщения за съвпадение.

index е място в локалната рамка, където се съхранява съответстващо съобщение.

Проверява указателя на съобщението, ако NIL (няма повече съобщения) се разклонява към инструкцията *Wait* или *WaitTOut* за спиране. В противен случай се записва съобщението в локалната променлива, *y(index)*, преди съвпадение. Инструкцията е последвана от други инструкции за необходимото съвпадение и инструкция *loopRecEnd*. Инструкцията *loopRecEnd* се поставя след всички инструкции, изпълняващи необходимото съответствие на шаблона.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_loopRecEnd

& loopRec

❑ *loopRecEnd*

Актуализиране на указателя на съобщение.

Синтаксис с нишков код

&loopRec е адрес на съответната инструкция loopRec.

Актуализира указателя на съобщението, разклонява се до съответната инструкция loopRec (опитайте следващото съобщение). Инструкцията се предхожда от съвпадаща инструкция loopRec.

lb_Wait

&rec

&FuncBegin

❑ *Wait*

Спиране на текущия процес на Erlang.

Синтаксис с нишков код

&rec е адрес на съответната инструкция Rec .

В пощенската кутия няма съответстващи съобщения и текущият процес на Erlang е спрял. Когато процесът е разсрочен за по-късно изпълнение, указателят на следващата инструкция е зададен на адреса &rec. При спиране се генерира текущата контекстна информация с помощта на &FuncBegin .

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_WaitTOut
&rec
&FuncBegin
type
time

❑ *WaitTOut*

Спиране на текущия процес на Erlang. Задаване на необходимото време за изчакване. Синтаксис с нишков код

&rec е адрес на съответната инструкция Rec.

type показва вида на следния операнд (X_arg, Y_arg, C_arg).

time съдържа необходимата стойност за изчакване.

Същата семантика като Wait с тази разлика, че първо се задава необходимото време за изчакване. Времето за изчакване се задава само веднъж, когато примитивът за получаване е въведен за първи път. Инструкцията е част от получаването, увеличено с таймаут.

lb_RmMsg

❑ *RmMsg*

Премахване на съобщение от пощенска кутия.

Синтаксис с нишков код

Премахване на съвпадащото съобщение от пощенската кутия на процеса. Инструкцията RmMsg се изпълнява, когато извлеченото съобщение е успешно съпоставено и съответното действие ще бъде оценено. Всички останали съобщения са маркирани като неизпробвани. Инструкцията изчиства предварително зададения таймаут (ако има такъв).

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_TimeOut

❑ *TimeOut*

Маркиране на всички съобщения в пощенската кутия като непрочетени.

Синтаксис с нишков код

Маркиране на всички съобщения в текущата пощенска кутия като непрочетени. Инструкцията TimeOut се изпълнява, когато настъпи таймаут и съобщенията трябва да бъдат маркирани като неизпробвани за бъдещи получавания.

lb_TPattEnd

&instruction

❑ *TPattEnd*

Разклоняване към даден адрес на инструкция.

Синтаксис с нишков код

&instruction е адрес на инструкция.

Задаване на указател за следваща инструкция на &instruction и разклоняването му. Инструкцията TPattEnd се използва за прехвърляне на контрол, когато една от алтернативите в receive/case/if примитивът е готов. &instruction сочи първата инструкция след примитива.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_CaseEnd
&FuncBegin
type
object

3.3.10. Примитиви (Primitives)

Единствените нови инструкции, необходими за прилагане на *case* и *if* семантиката са: *TPattEnd* (обхванати по-рано в контекста на *receive*), *CaseEnd* и *IfEnd*.

□ *CaseEnd*

Служи за генериране на грешката *CASE_CLAUSE* (няма съответстващи шаблони на регистър).

Синтаксис с нишков код

type показва вид от следния операнд (*X_arg*, *Y_arg*, *C_arg*).

object е вътрешен обект от данни, който се съпоставя.

Инструкцията се изпълнява, когато няма съвпадащи шаблони на регистър. Генерира се текущата контекстна информация с помощта на *&FuncBegin*, задава се грешката *CASE_CLAUSE*, генерира се информация за обекта, който се съпоставя, разклонява се до процедура за обработка на грешки.

lb_IfEnd
&FuncBegin

□ *IfEnd*

Генериране на грешката *IF_CLAUSE* (тя е неуспешна, ако има защита).

Синтаксис с нишков код

Генерира текущата контекстна информация с помощта на *&FuncBegin*, задава *IF_CLAUSE* грешка, и се разклонява към процедура за обработка на грешки.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_Bif_0_<type>
bif
ret

3.3.11. Вградени функции

Вградените функции се компилират в различни инструкции в зависимост от функционалността. От съображения за ефективност някои вградени функции се кодират изрично. Има някои други изрази на Erlang, които се третират като вградени функции (например смесена аритметика, целочислена аритметика).

□ *Bif_0_<type>*

Извиква вградена функция от arity 0 (функцията не може да се провали).

Синтаксис с нишков код

bif е bif-table индекс.

ret е вътрешен обект от данни.

Като се има предвид индекс на bif-table, **bif**, изпълнява съответната вградена функция. След това се поставя резултата в обекта **ret**. Типът на обекта (*X_arg* или *Y_arg*) се кодира имплицитно като код на инструкцията **opcode**.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_Bif_1_<type>
address
bif
arg
ret

□ *Bif_1_<type>*

Извиква вградена функция от arity 1 (функцията може да се провали).

Синтаксис с нишков код

bif е bif-table индекс.

address е адрес на разклонение в случай на повреда.

arg и **ret** са вътрешни обекти от данни.

Като се има предвид индекс на bif-table, bif и аргумент, arg, се изпълнява съответната вградена функция. Поставя се резултата в обекта ret. В случай на неуспех, ако вградената функция се появи в защита, се разклонява към следващата clause/option, използвайки address. Ако функцията се среща в тялото (адресът сочи към съответния FuncBegin), се генерира текущата контекстна информация и се извиква процедурата за обработка на грешки. Информацията за грешка се задава от неуспешната вградена функция. Видовете arg и ret се кодират имплицитно като код на инструкцията opcode.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_Bif_2
address
bif
type
arg0
type
arg1
type
ret

□ *Bif_2*

Извиква вградена функция от arity 2 (функцията може да се провали).

Синтаксис с нишков код

Bif e bif-table индекс.

address е *адрес* на разклонение в случай на повреда.

type показва вида на следния операнд.

arg0, *arg1* и *ret* са вътрешни обекти от данни.

Същата семантика като *Bif_1_<type>* с тази разлика, че вградената функция се извиква с два аргумента и изрично са зададени типове на всички операнди.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_Bif_3
address
bif
type
arg0
type
arg1
type
arg2
type
ret

□ *Bif_3*

Извиква вградена функция от arity 3 (функцията може да се провали). Функцията може да се появи както в clause/case/if/receive глава (смесена аритметика, целочислена аритметика) или тяло.

Синтаксис с нишков код

bif е bif-table индекс.

address е адрес на разклонение в случай на повреда.

type показва вид на следния операнд.

arg0 , *arg1* , *arg2* и *ret* са вътрешни обекти от данни.

Като се има предвид индекс на bif-table, *bif* и аргументи: *arg0* , *arg1* и *arg2* изпълняват съответна вградена функция. Резултата се поставя в обекта *ret*. В случай на неуспех, ако вградената функция е защитена, се разклонява към следващата clause/option, използвайки *address*. Ако функцията се среща в тяло (адресът сочи към съответния *FuncBegin*), се генерира текущата контекстна информация и се извиква процедурата за обработка на грешки. Информацията за грешка се задава от неуспешната вградена функция.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_Appl3
&FuncBegin
type
module
type
func
type
arg_list

□ *Appl3*

Извиква вградена функция *apply/3* (функцията може да се провали). Функцията може да се появи само в *clause/case/if/receive* тяло.

Синтаксис с нишков код

&FuncBegin е адрес на текущата функция **FuncBegin** инструкция.

type показва вида на следния операнд.

module, **func** и **arg_list** са вътрешни обекти от данни.

Прилагане при повикване (модул, функция, **arg_list**). Поставяне се резултата в **x(0)**. В случай на неуспех, се генерира текущата контекстна информация и се насочва към процедурата за обработка на грешки. Информацията за грешка се задава от неуспешната вградена функция.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_Hd/Tl
address
type
list
type
hd/tl

□ *Hd/Tl*

Извиква вградена функция *hd/1* или *tl/1* (функцията може да се провали).

Синтаксис с нишков код

address е адрес на разклонение в случай на повреда.

type показва вида на следния операнд.

list и **hd/tl** са вътрешни обекти на данни.

Като има списък, *list*, се извлича *head/tail* му. Поставя се резултата в *hd/tl* обекта. В случай на неуспех, ако вградената функция е защитена, се разклонява към следващата *clause/option*, използвайки *address*. Ако функцията се среща в тяло (адресът сочи към съответния *FuncBegin*), се генерира текущата контекстна информация и се извиква процедурата за обработка на грешки. Информацията за грешка се задава от неуспешната вградена функция.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_ <Bif0>
type
ret

❑ <Bif0>

Извиква дадена вградена функция (функцията не може да се провали).

<Bif0> може да бъде:

- *CurrentNode* - *node/0*
- *Self* - *self/0*
- *MakeRef* - *make_ref/0*
- *Gleader* - *group_leader/0*
- *GrbCollect* - *garbage_collect/0*
- *ErAll-erase/0*

Синтаксис с нишков код

type показва вида на следния операнд.

ret е вътрешен обект от данни.

Извиква дадена вградена функция. Поставя резултата в обекта **ret**.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_CheckPrCode/ExitPid
&FuncBegin
type
arg0
type
arg1
type
ret

❑ *CheckPrCode/ExitPid*

Извиква `check_process_code/2` или `exit/2` вградена функция (функцията може да се провали). Функцията може да се появи само в `clause/case/if/receive` тяло.

Синтаксис с нишков код

&FuncBegin е адрес на текущата функция `FuncBegin` инструкция.

type показва вида на следния операнд.

arg0, **arg1** и **ret** са вътрешни обекти от данни.

Извиква `check_process_code(arg0, arg1)` или `exit(arg0, arg1)`. Поставя резултата в `ret`. В случай на неуспех, генерира текущата контекстна информация и се насочва към процедурата за обработка на грешки. Информацията за грешка се задава от неуспешната вградена функция.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_Exit
&FuncBegin
type
arg0

☐ **Exit**

Извиква *exit/1* вградена функция.

Синтаксис с нишков код

&FuncBegin е адрес на текущата функция FuncBegin инструкция.

type показва вида на следния операнд.

arg0 е вътрешен обект от данни.

Извиква *exit(arg0)*. **&FuncBegin** се използва за генериране на необходимата контекстна информация.

lb_GrbCollect
&FuncBegin
type
arg0
type

☐ **GrbCollect**

Извиква вградена функция *garbage_collect/1* (функцията може да се провали). Функцията може да се появи само в *clause/case/if/receive* тяло.

Синтаксис с нишков код

&FuncBegin е адрес на текущата функция FuncBegin инструкция.

type показва вида на следния операнд.

arg0 и **ret** са вътрешни обекти от данни.

Извиква *garbage_collect(arg0)*. Постава резултата в **ret**. В случай на неуспех, генерира текущата контекстна информация и се насочва към процедурата за обработка на грешки. Информацията за грешка се задава от неуспешната вградена функция.

lb_Halt

Halt

Извиква вградена функция *halt/0*.

Синтаксис с нишков код

Извиква *halt ()*.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_FlArth
address
op
type
arg0
type
arg1
type
ret

3.3.12. Аритметични изрази

Аритметичните изрази се компилират в извиквания на C-функция с общо предназначение, изпълняващи необходимата аритметика, и се третират като извиквания към вградени функции, обхванати вече в контекста на *вградените функции*. От съображения за ефективност (когато анализът на времето на компилация генерира повече информация за дадените аргументи) някои аритметични изрази се кодират изрично.

□ *FlArth*

Предназначен за оценяване на даден аритметичен израз, чийто аргументи са числа с плаваща запетая.

Синтаксис с нишков код

address е *адрес* на разклонение в случай на повреда.

op е зададен оператор (+, -, *).

type показва вида на следния операнд.

arg0, *arg1* и *ret* са вътрешни обекти от данни.

Наличието на оператор *op* и неговите аргументи *arg0* и *arg1*, оценяват необходимата аритметика с плаваща запетая. Поставя се резултата в *ret*. В случай на неуспех, ако аритметичният израз се появи в защитата, става разклоняване към следващата *clause/option*, използвайки *address*. Ако изразът се среща в тяло (*адресът* посочва съответния *FuncBegin*), се генерира текущата контекстна информация, *задава се* грешката *BADARITH* и се извиква процедурата за обработка на грешки.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_FlArthDiv
address
type
arg0
type
arg1
type
ret

□ *FlArthDiv*

Оценява се разделяне с плаваща запетая, като аргументите са числа с плаваща запетая. Синтаксис с нишков код

address е адрес на клон в случай на повреда.

type показва вида на следния операнд.

arg0, **arg1** и **ret** са вътрешни обекти от данни.

Когато имаме два аргумента с плаваща запетая, **arg0** и **arg1**, се оценява разделението с плаваща запетая. Поставете резултата в **ret**. В случай на неуспех, ако аритметичният израз се появи в защитата, разклонете се към следващата clause/option, използвайки **address**. Ако изразът се среща в тяло (**address** посочва съответния **FuncBegin**), се генерира текущата контекстна информация, задава се грешката **BADARITH** и се извиква процедурата за обработка на грешки.

lb_IntArth
op
type
arg0
type
arg1
type
ret

□ *IntArth*

Оценява се даден аритметичен израз, аргументите са цели числа (не може да се провали). Синтаксис с нишков код

op е зададен оператор (+, -, *).

type показва вида на следния операнд.

arg0, **arg1** и **ret** са вътрешни обекти от данни.

Наличието на оператор, **op** и неговите цели аргументи **arg0** и **arg1**, оценяват необходимата аритметика. Резултата се поставя в **ret**.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_IntArth_<op>_<type>
arg0
arg1

□ *IntArth_<op>_<type>*

Оценяване на даден аритметичен израз, чийто аргументи са цели числа. *IntArth_<op>_<type>* е специализирана версия на инструкцията *IntArth*.

Синтаксис с нишков код

arg0, **arg1** и **ret** са вътрешни обекти от данни.

Наличието на оператор и неговите цели аргументи **arg0** и **arg1**, оценяват необходимата аритметика. Резултата се поставя в **ret**. Операторът и типовете операнди се кодират имплицитно като код на инструкцията **opcode**.

lb_Plus/Minus_<type>
address
arg0
arg1
ret

□ *Plus/Minus_<type>*

Оценяване на даден аритметичен израз, чийто втори аргумент е малко цяло число.

Синтаксис с нишков код

address е адрес на разклонен в случай на повреда.

arg1 е small integer.

arg0 и **ret** са вътрешни обекти от данни.

Наличието на оператор и неговите аргументи (вторият аргумент е small integer) проверете дали резултатът е small integer (като проверите първия аргумент). Ако не, извикайте **C**-функция с общо предназначение, която извършва аритметиката. Ако резултатът е успешен, оценете аритметичния израз директно в емулятора. Резултата се поставя в **ret**. В случай на неуспех, ако аритметичният израз се появи в защитата, става разклонение към следващата **clause/option**, използвайки **address**. Ако изразът се среща в тялото (**address** посочва съответния **FuncBegin**), се генерира текущата контекстна информация, задава се грешката **BADARITH** и се извиква процедурата за обработка на грешки. Видовете **arg0** и **ret** се кодират имплицитно като код на инструкцията **opcode**.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_ArthBnot
address
type
arg0
type
ret

□ *ArthBnot*

Оценяване побитово, а не аритметично.

Синтаксис с нишков код

address е адрес на разклонение в случай на повреда.

type показва вида на следния операнд.

arg0 и **ret** са вътрешни обекти от данни.

Ако имаме аргументи, **arg0**, се оценява побитово, а не аритметично. Резултата се поставя в **ret**. В случай на неуспех, ако аритметичният израз се появи в защитата, става разклонение към следващата **clause/option**, използвайки **address**. Ако изразът се среща в тялото (**address** посочва съответния **FuncBegin**), се генерира текущата контекстна информация, задава се грешката **BADARITH** и се извиква процедурата за обработка на грешки.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_Cmp/IntCmp/FnCmp
address
cmp
type
arg0
type
arg1

3.3.13. Term Comparisons

Сравненията на терми в guard (Предпазителите са начин за увеличаване на съвпадението на шаблона с по-сложни проверки. Те са разрешени в предварително дефиниран набор от конструкции, където е разрешено съвпадение на шаблони) са компилирани в инструкции с общо предназначение, които правят необходимото сравнение. От съображения за ефективност, когато дадени аргументи могат да бъдат типово декорирани от анализа на времето за компилиране, сравненията се кодират изрично. Някои сравнения на терми, които възникват в clause/case/if/receive тяло, се компилират в извиквания на съответните функции в модула *erlang* и се третират съответно.

□ *Cmp/IntCmp/FnCmp*

Оценява се сравнение на даден term. Инструкцията се среща само при guard.

Cmp - когато типовете аргументи са неизвестни; *IntCmp* - когато аргументите са цели числа и *FnCmp* - когато аргументите са плаващи.

Синтаксис с нишков код

address е адрес на клон в случай на повреда.

cmp е оператор за сравнение на терми (Cmp:>, <,> =, = <, ==, /=; IntCmp /FnCmp:>, <,> =, = <)

type показва вида на следния операнд.

arg0 и *arg1* са вътрешни обекти от данни.

При наличието на оператор за сравнение cmp и неговите аргументи, arg0 и arg1, се извършва необходимото сравнение на терми. В случай на неуспех, преминете към следващата clause/option, използвайки address.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_ExEqEq_head/Eq_head/ExNEq/NEql
address
type
arg0
type
arg1

❑ *ExEqEq_head/Eq_head/ExNEq/NEql*

Оценяване на сравнение на даден term. Инструкцията се среща само в clause/case/if/receive head или охрана. Използват се както следва: *ExEqEq_head* - когато е точно равно; *ExNEq* - когато точно не е равно. *Eql_head* е специализирана версия на инструкцията *ExEqEq*, а *NEql* е специализирана версия на инструкцията *ExNEq*, когато аргументите са small integers или атоми.

Синтаксис с нишков код

address е адрес на клон в случай на повреда.

type показва вида на следния операнд.

arg0 и **arg1** са вътрешни обекти от данни.

Като имаме два аргумента, arg0 и arg1, се извършва необходимото сравнение на терми (*ExEqEq_head* : =: =; *ExNEq* : =/=). Когато аргументите са small integer или атоми, точното неравно сравнение, *NEql* и точно същото сравнение, *Eql*, се правят директно в емулятора. В случай на неуспех, се преминава към следващата clause/option, като се използва address.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_Eql_head_<type>
address
arg0
arg1

□ *Eql_head_<type>*

Оценява се точното сравнение с еднакви срокове. *Eql_head_<type>* е допълнителна специализация на инструкцията *Eql_head*.

Синтаксис с нишков код

address е адрес на разклонение в случай на повреда.

arg0 и **arg1** са вътрешни обекти от данни.

Eql_head_<type> има същата семантика като инструкцията *Eql_head*. Единствената разлика е, че типовете на всички операнди се кодират имплицитно като код за инструкции. Кой видове са кодирани като специализирани инструкции се основава на честотата им в съществуващите големи промишлени приложения.

lb_ExEqEq/Eql
&FuncBegin
type
arg0
type

□ *ExEqEq/Eql*

Оценява се точното сравнение с еднакви срокове. *Eql* е специализирана версия на инструкцията *ExEqEq* (когато аргументите са малки цели числа или атоми). Инструкциите се срещат само в *clause/case/if/receive* тяло.

Синтаксис с нишков код

&FuncBegin е адрес на текущата функция *FuncBegin* инструкция.

type показва вида на следния операнд.

arg0 и **arg1** са вътрешни обекти от данни.

Когато имаме два аргумента, *arg0* и *arg1*, се изпълнява точно равно `= =:`, сравнение на *term*-и. Когато аргументите са малки цели числа или атоми, сравнението се извършва директно в емулятора (*Eql*). В случай на неуспех, се генерира текущата контекстна информация, като се използва *&FuncBegin*, и се задава грешката *BADMATCH*, и се генерира информация за обекта, който се съпоставя, и се разклонява процедурата за обработка на грешки.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_LookupSwitch
type
arg0
size
value0
address0
...
valueN

3.3.14. Инструкции за индексирание

Erlang BEAM обработва индексирането имплицитно, като изгражда дърво за вземане на решения за достъп до различни клаузи или опции (избягвайки повторения на едни и същи тестове), и изрично чрез набор от инструкции за индексирание на цели и атомни стойности.

□ *LookupSwitch*

LookupSwitch се използва за индексирание на цели числа.

Синтаксис с нишков код

type показва вида на следния операнд.

arg0 е вътрешен обект от данни.

size е размер на включената таблица за скокове (N+1).

value <i> е цяло число.

address <i> е адрес на разклонение в случай на съвпадение на стойност.

След като **arg0** провери стойността му спрямо стойностите в дадена таблица за скок, в случай на съвпадение при разклонение със съответната инструкция (използвайки адрес <i>), за да се даде възможност за двоично търсене, стойностите се сортират, когато се генерира инструкцията. Ако няма намерени съвпадения, се избира следващата инструкция.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

<i>lb JumpSwitch</i>
<i>type</i>
<i>arg0</i>
<i>size</i>
<i>address0</i>
...
<i>addressN</i>
<i>lb JumpSwitch</i>

□ *JumpSwitch*

JumpSwitch е специализирана версия на инструкцията *LookupSwitch*, когато целочислените стойности изграждат последователност: 0, 1, 2, 3, ..., N.

Синтаксис с нишков код

type показва вид на следния операнд.

arg0 е вътрешен обект с данни.

size е размерът на включената таблица за скачане (N+1).

address<i> е адрес на клон в случай на съвпадение на стойност.

Когато *arg0* провери стойността му спрямо размера на дадена таблица за скок, ако стойността е по-малка или равна на дадения размер, се използва съответният *address* на клон, иначе, се избира следващата инструкция.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

AtomSwitch_op
type
arg0
size
atom_name0
offset0
...
atom_nameN
offsetN

lb_AtomSwitch
type
arg0
size
atom0
address0
...
atomN

□ *AtomSwitch*

AtomSwitch се използва за индексване на атомни стойности.

Синтаксис с байтов и с нишков код

type показва вида на следния операнд.

arg0 е вътрешен обект от данни.

size е размер на включената таблица за скокове (N+1).

atom<i> е готов създаден атом.

address <i> е адрес на клон в случай на съвпадение на стойност.

След като arg0 провери за атомите в дадена таблица за скок, в случай на съвпадение на клон със съответната инструкция (се използва **address <i>**). За да се даде възможност за двоично търсене, атомите се сортират, когато инструкцията е заредена и атомите са изградени. Ако няма намерени съвпадения, се избира следващата инструкция.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_AH
frame_size
m
n

3.3.15. Сгъване на инструкции (Instruction Folding)

За да се спести място и да се подобри времето за емуляция (декодиране на инструкции, актуализиране на указателя за следваща инструкция, I) някои много чести последователности на инструкции се кодират като една инструкция. Решението кои последователности от инструкции да бъдат кодирани изрично се основава на тяхната честота в съществуващите големи промишлени приложения.

□ *AH*

Разпределя се нова рамка на стека и се тества за необходимото пространство за hear. *AH* е резултат от сгъване на инструкции от *A* и *TH* инструкции.

Синтаксис с нишков код

m е размер на необходимото пространство за hear (с думи).

n е брой живи временни променливи $x(0), \dots, x(n-1)$ (за събиране на боклук).

Извършва се проверка за необходимия стек и hear пространство, обаждаме се за събиране на боклука, ако е необходимо. Разпределя се нова рамка на стека. Актуализира се stack-top на указател *E*. Копира се указателя *CP* на програмата за продължаване в новия кадър.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_PT_<type>
tuple
arity
element

□ *PT_<type>*

Инструкцията *PT_<type>* е резултат от сгъване на инструкции на инструкциите *PT0_<type>* и *PTEl_<type>*. Може да бъде последва от редица инструкции за *PTEl_<type>*.

Синтаксис с нишков код

tuple е указател към създавания обект с данни кортеж.

arity дава номер на следните кортежни елементи.

element е кортежният първи елемент.

Създава се начална структура за обект с данни кортеж. Изгражда се първия кортежен елемент. Задава се указател на *кортежа* към новосъздадения обект. Актуализира се *heap-top* указателя. Типът *кортеж* неявно се кодира имплицитно като код на инструкцията *opcode*.

lb_M_R_<type>r object

□ *M_R_<type>r*

Премества се вътрешен обект от данни в регистъра *x (0)*. Връщане от местно извикване на функцията *Erlang*.

Синтаксис с нишков код

object е вътрешен обект с данни.

Преместване на вътрешен обект от данни, обект (може да бъде от тип *X_arg*, *Y_arg*, *C_arg*) към *x (0)* и връщане на контрола на повикващия функцията.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_M_D_R_<type>r

object

frame_size

M_D_R_<type> r

Преместване на вътрешен обект от данни в регистъра *x* (*0*). Изхвърляне на текущата рамка на стека. Връщане от местно извикване на функция Erlang.

Синтаксис с нишков код

object е вътрешен обект с данни.

frame_size е размер на текущата рамка на стека, която трябва да се изхвърли.

Преместване на вътрешен обект от данни, обект, до *x*(*0*). Копиране на указателя на програмата за продължаване от текущата рамка на стека в СР и изхвърляне на рамката. Актуализиране на stack-top на указателя Е. Връщане на контрола на извикващата функция.

lb_D_R

frame_size

D_R

Изхвърляне на текущата рамка на стека. Връщане от местно извикване на функция Erlang.

Синтаксис с нишков код

frame_size е размер на текущата рамка на стека, която трябва да се изхвърли.

Копиране на показалеца на програмата за продължаване от текущата рамка на стека в СР и изхвърляне на рамката. Актуализиране на **комин** на върха на показалеца Е. Връщане на контрола на извикващата функция.

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb TT GTp_<type>
&FuncBegin
object
arity

□ *TT GTp_<type>*

Проверка за това дали даден обект от данни на Erlang е кортеж от необходимата Arity. Синтаксис с нишков код

&FuncBegin е адрес на текущата функция FuncBegin инструкция.

object е съществуващ обект от данни на Erlang.

arity е необходимото arity.

Инструкцията се среща само в clause/case/if/receive тяло. При наличието на даден обект от данни се проверява дали това е набор от arity. В случай на неуспех, се генерира текущата контекстна информация, задава се грешката BADMATCH и се разклонява процедурата за обработка на грешки. Типът object се кодира имплицитно като код на инструкция opcode.

□ *TT GTp_head_<type>*

Проверява се дали даден обект от данни на Erlang е кортеж от необходимото Arity. *TT GTp_head_<type>* е специализирана версия на инструкцията *TT GTp_<type>*. Синтаксис с нишков код

address е адрес на клон в случай на повреда.

object е съществуващ обект от данни на Erlang.

arity е необходимата arity.

Инструкцията се среща само в clause/case/if/receive глава. При наличие на даден обект от данни се проверява дали това е набор от arity. В случай на неуспех се насочваме към следващата clause/option, като се използва address. Типът object се кодира имплицитно, като код на инструкция opcode.

lb TT GTp_head_<type>
address
object
arity

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_TNonEmpL/TInt_ A_<type>
address
object
frame_size
n

□ *TNonEmpL_A_<type>/TInt_A_<type>*

Проверка дали даден обект от данни Erlang не е празен списък/цяло число. Ако условието е успешно, се разпределя нова рамка на стека.

Синтаксис с нишков код

address е адрес на разклонение в случай на повреда.

object е съществуващ обект от данни на Erlang.

frame_size е необходимия размер на рамката.

n е брой живи временни променливи $x(0), \dots, x(n-1)$ (за събиране на боклук).

Инструкцията се среща само в clause/case/if/receive head. При наличие на определен обект от данни, се извършва проверка дали той не е празен списък/цяло число. При true, се разпределя нова рамка на стека (на frame_size). При false, се преминава към следваща clause/option, като се използва address. Типът object се кодира имплицитно, като код на инструкцията opcode.

□ *I_<n>*

Инициализира редица локални променливи ($n = 2/3/4$).

Синтаксис с нишков код

index <i> е индекс на локална променлива $y(\text{index}<i>)$, който трябва да бъде изрично инициализиран.

Инициализира редица локални променливи $y(\text{index}<i>)$. Поради някои оптимизации на компилатора индексите не са непременно последователност от числа.

lb_I_<n>
index0
...
indexN

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

lb_A_I_<n>
frame_size
n
index0
...
indexN

□ **A_I_<n>**

Разпределя нова рамка на стека. Инициализира редица локални променливи ($n = 1/2/3/4$).

Синтаксис с нишков код

frame_size е необходимия размер на рамката.

n е брой живи временни променливи $x(0), \dots, x(n-1)$ (за събиране на боклук).

index<i> е индекс на локална променлива $y(\text{index } <i>)$, който трябва да бъде изрично инициализиран.

Разпределя нова рамка на стека (на **frame_size**). Инициализира редица локални променливи $y(\text{index } <i>)$. Поради някои оптимизации на компилатора индексите не са непременно последователност от числа.

M_C_<type>r_op
object
function name
function arity

□ **M_C_<type> r**

Зарежда регистъра на аргументите $x(0)$. Обръща се към на резидентна функция Erlang.

Синтаксис на байтов код и нишков код

object е вътрешен обект с данни.

&function е адрес на първата инструкция, съответстваща на извиканата функция.

function_arity е брой аргументи на функцията.

&FuncBegin е адрес на текущата функция *FuncBegin* инструкция.

Преместване на вътрешен обект от данни, *object*, до $x(0)$. Задаване на указателя на програмата за продължаване при следващата инструкция, увеличава брояча на редукции и проверка за прекъсване (ако спре, се генерира текущата контекстна информация), разклонение към извикана функция. Типът *object* се кодира имплицитно, като код на инструкцията opcode.

lb_M_C_<type>r
object
&function
function_arity
&FuncBegin

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

M_CL_<type>r_op

object

function name

function arity

frame size

□ *M_CL_<type> r*

Зарежда регистъра на аргументите $x(0)$. Изхвърля текущата рамка на стека. Извиква локална функция Erlang.

Синтаксис на байтов код

object е вътрешен обект с данни.

frame_size е размер на текущата рамка на стека, която трябва да се изхвърли.

Премества вътрешен обект от данни, *object*, до $x(0)$. Копира указателя на програмата за продължаване от текущата рамка на стека в CP, изхвърля рамката, увеличава брояча на редукции и проверява за прекъсване, разклонява се към извиканата функция. Типът *object* се кодира имплицитно, като код на инструкция *opcode*.

lb_M_CL_<type>r

object

&function

function_arity

frame_size

&FuncBegin

Тема 2. Erlang и Elixir. Характеристика и особености на BEAM виртуалната машина. Разпределение на паметта, начин на извод.

Използвана литература

- [1] Bogumil Hausman, 11.1997, “*The Erlang BEAM Virtual Machine Specification*“, Computer Science Laboratory, Ericsson Telecom AB, Rev. 1.2, // http://www.cs-lab.org/historical_beam_instruction_set.html - последно достъпен 31.01.2022
- [2] John Högberg, 20.11.2020 // <https://blog.erlang.org/a-brief-BEAM-primer/> - последно достъпен 31.01.2022
- [3] <https://hexdocs.pm/elixir/guards.html> - последно достъпен на 20.02.2022